# Experiment No.4

**AIM**: To set up a Continuous Integration Pipeline using Jenkins/GitHub Act.

**THEORY:**

**Continuous Integration**

Continuous Integration (CI) is a development practice that promotes collaboration, early issue detection, and automated testing to enhance the quality and efficiency of software development.

In Continuous Integration, developers frequently integrate their code changes into a shared project. The integrated code can then be automatically built and tested, helping identify errors at an early stage of the development process.

**What is Jenkins?**

Jenkins is an open-source automation server that facilitates Continuous Integration and Continuous Delivery (CI/CD) of software projects. It helps developers automate various stages of the software development lifecycle, from building and testing to deployment and monitoring. Jenkins supports a wide range of plugins and integrations, making it a versatile tool for automating various tasks in software development. Jenkins can be used to automate repetitive processes so that software development becomes more systematic, consistent, and efficient.

**What is a Pipeline in Jenkins?**

A Jenkins Pipeline is a set of automated processes that help developers and DevOps teams build, test, and deploy code in a continuous integration/continuous delivery (CI/CD) environment. In simple terms, it is a workflow that defines how software moves from code to production in an automated and repeatable manner. A Pipeline allows the different activities involved in software development to be represented as a sequence of stages. These stages can include building the application, testing the code, packaging it, and deploying it. The pipeline is defined using a Jenkinsfile, which contains the instructions that Jenkins follows during execution.

**Key Benefits of Jenkins Pipeline**

The major benefits of using Jenkins Pipeline are:

• Automates CI/CD processes

Jenkins Pipeline automates different activities involved in continuous integration and continuous delivery. This reduces the need for performing repetitive tasks manually.

• Provides better visibility into the build process

The pipeline divides the workflow into different stages. This makes it easier to observe the progress of the build and identify where an error has occurred.

• Offers version control of build logic — Pipeline as Code

The pipeline configuration can be written in a Jenkinsfile and maintained along with the project source code. This provides version control for the build and automation logic.

• Supports complex workflows

Jenkins Pipelines can support advanced workflows such as parallel execution, conditional logic, and multiple stages, allowing complex software development processes to be automated.

**Types of Jenkins Pipelines**

There are two main types of syntax used to define Jenkins pipelines:

1. Declarative Pipeline
2. Scripted Pipeline

**Declarative Pipeline**

A Declarative Pipeline uses a simpler and more structured syntax.

It is designed for users who want an easy-to-read format and was introduced later to make pipelines more readable and maintainable.

Key Features are:

- Uses the pipeline {} block.
- Organized into stages and steps.
- Provides easier error handling.
- Provides better validation in the Jenkins UI.

Because of its structured syntax, the Declarative Pipeline is generally easier for beginners to understand and maintain.

**Scripted Pipeline**

A Scripted Pipeline is more flexible and powerful compared to a Declarative Pipeline.

It is written using Groovy-based syntax and gives the developer full control over the build process.

It is suitable for advanced users who need to implement complex scripting and logic.

Key Features

- Uses the node {} block.
- Everything is written as code.
- Allows complex scripting and logic.
- Is less user-friendly for beginners.

Scripted Pipelines provide greater flexibility when implementing complicated build processes, but their syntax can be more difficult for beginners to understand.

**Jenkins Installation**

The document describes the following basic steps for installing and setting up Jenkins.

**Step 1 — Java Installation**

Ensure that the Java Development Kit (JDK) is installed on the system.

Java is required for Jenkins to run.

The Java installation can be verified from the terminal using:

*java -version*

This displays the installed Java version.

**Step 2 — Download Jenkins**

The Jenkins WAR file can be obtained from the official Jenkins website or through the appropriate package source.

The WAR file is used to run the Jenkins server.

**Step 3 — Launch Terminal**

Open a terminal or Command Prompt on the system.

The document demonstrates launching Jenkins using the Java command.

**Step 4 — Navigate to Directory**

Use the cd command to navigate to the directory containing the Jenkins WAR file.

For example:

*cd <Jenkins-WAR-directory>*

This ensures that the terminal is working in the directory where the Jenkins WAR file is located.

**Step 5 — Start Jenkins**

Jenkins can be started using:

*java -jar jenkins.war*

Once the command is executed, Jenkins starts its server and performs its initial setup.

**Step 6 — Access Web UI**

After starting Jenkins, open a web browser and navigate to:

*http://localhost:8080*

This opens the Jenkins setup wizard.

**9. Unlock Jenkins**

During the initial setup, Jenkins asks for the initial administrator password.

The password is generated automatically when Jenkins is started for the first time.

The terminal displays the location of the password file.

The initial administrator password can then be retrieved from the specified directory and entered the Unlock Jenkins page.

**10. Customize Jenkins**

After entering the correct password, Jenkins redirects the user to the Customize Jenkins page.

At this stage, the user can select the plugins to be installed.

The document mentions choosing:

***Install suggested plugins***

or selecting the required plugins manually.

Plugins extend the functionality of Jenkins and allow it to work with different development and automation tools.

**11. Create Admin User**

After the plugin installation, Jenkins provides the option to create an administrator user.

The user needs to provide appropriate credentials such as:

- Username
- Password
- Full name
- Email address

These credentials are then used to access the Jenkins dashboard.

**12. Jenkins URL**

Jenkins then provides the option to confirm the Jenkins URL.

For a local installation, the URL is generally:

http://localhost:8080

The URL is used to access the Jenkins web interface from a browser.

**13. Jenkins Dashboard**

After completing the setup process, the user can log in using the administrator credentials.

The Jenkins Dashboard provides access to Jenkins jobs, pipelines, builds, and other configuration options.

From the dashboard, users can create and manage Jenkins projects and pipelines.

**14. Pipeline Execution**

After creating the Jenkins Pipeline, Jenkins executes the stages defined in the Jenkinsfile.

For the pipeline used in this experiment, the execution flow is:

Pipeline

↓

Agent

↓

Stages

↓

Hello Stage

↓

Steps

↓

echo "Hello, Jenkins!"

The successful execution of the pipeline produces the message:

Hello, Jenkins!

in the console output.

The build is considered successful when Jenkins displays:

Finished: SUCCESS

**OUTPUT:**

```
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\yogini bhatia>java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Temurin-21.0.12+8 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12+8 (build 21.0.12+8-LTS, mixed mode, sharing)

C:\Users\yogini bhatia>javac -version
javac 21.0.12

C:\Users\yogini bhatia>echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot\

C:\Users\yogini bhatia>where java
C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot\bin\java.exe
C:\Program Files\Eclipse Adoptium\jdk-17.0.16.8-hotspot\bin\java.exe
C:\Program Files\Common Files\Oracle\Java\javapath\java.exe

C:\Users\yogini bhatia>setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-21.0.x-hotspot"

SUCCESS: Specified value was saved.

C:\Users\yogini bhatia>
```

```
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\yogini bhatia>java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Temurin-21.0.12+8 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12+8 (build 21.0.12+8-LTS, mixed mode, sharing)

C:\Users\yogini bhatia>javac -version
javac 21.0.12

C:\Users\yogini bhatia>echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot\

C:\Users\yogini bhatia>where java
C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot\bin\java.exe
C:\Program Files\Eclipse Adoptium\jdk-17.0.16.8-hotspot\bin\java.exe
C:\Program Files\Common Files\Oracle\Java\javapath\java.exe

C:\Users\yogini bhatia>setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-21.0.x-hotspot"

SUCCESS: Specified value was saved.

C:\Users\yogini bhatia>
```

```
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\yogini bhatia>echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-21.0.x-hotspot

C:\Users\yogini bhatia>echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-21.0.x-hotspot

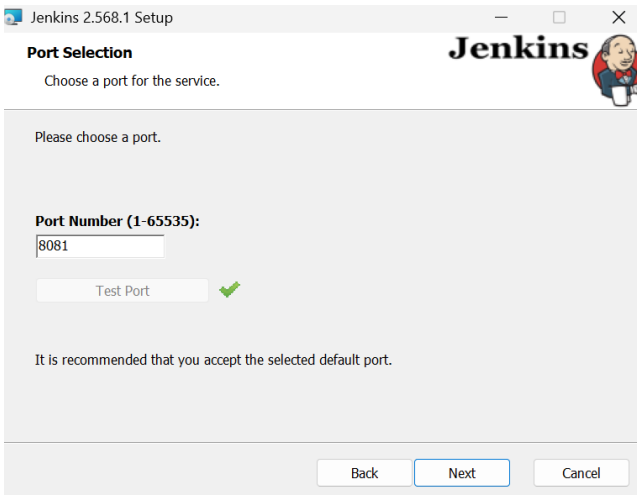
C:\Users\yogini bhatia>java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Temurin-21.0.12+8 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12+8 (build 21.0.12+8-LTS, mixed mode, sharing)

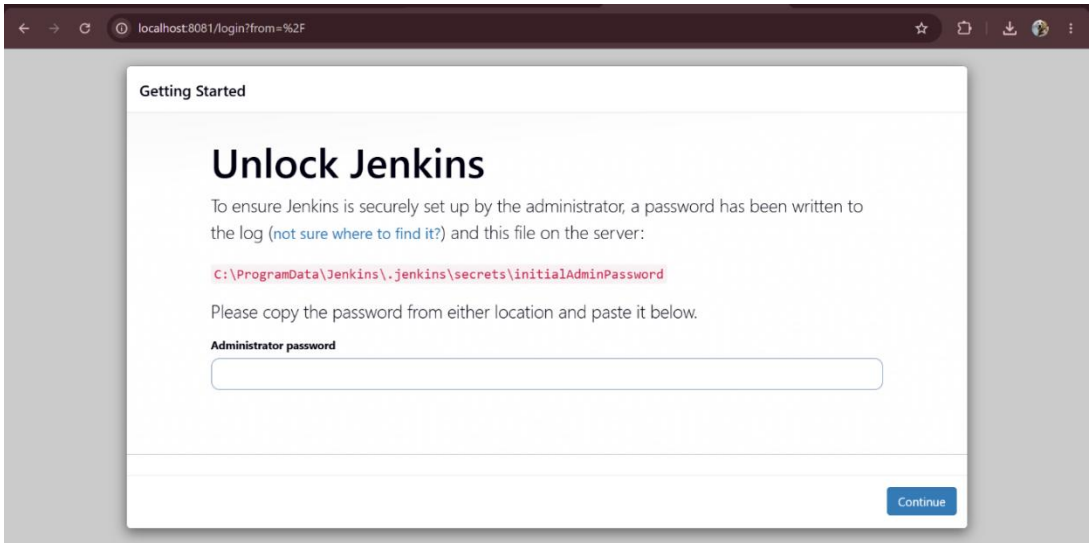
C:\Users\yogini bhatia>javac -version
javac 21.0.12

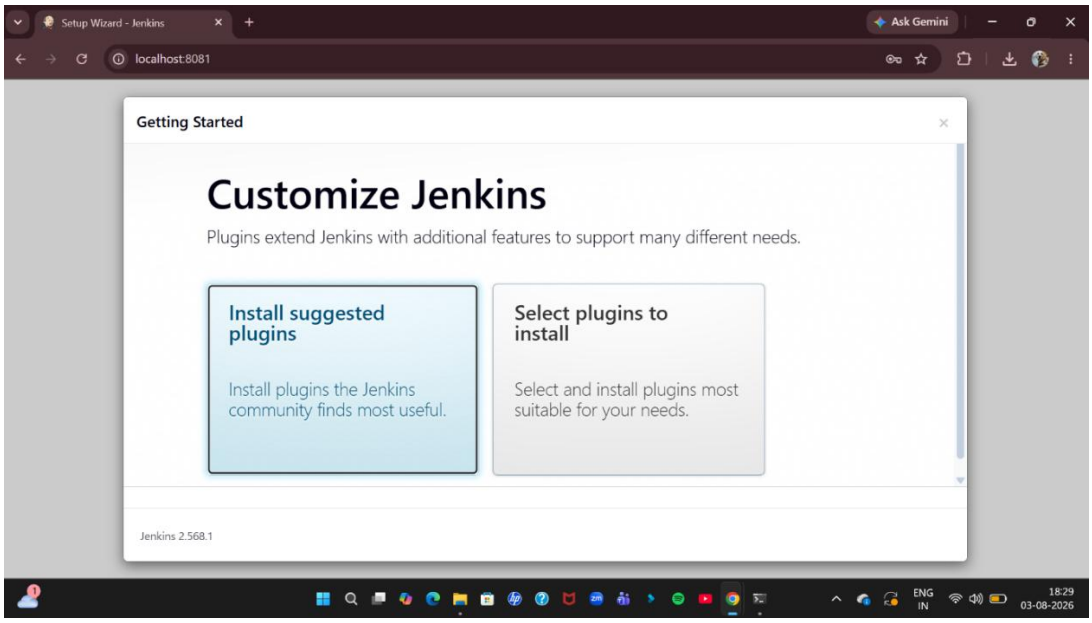
C:\Users\yogini bhatia>git --version
git version 2.50.1.windows.1

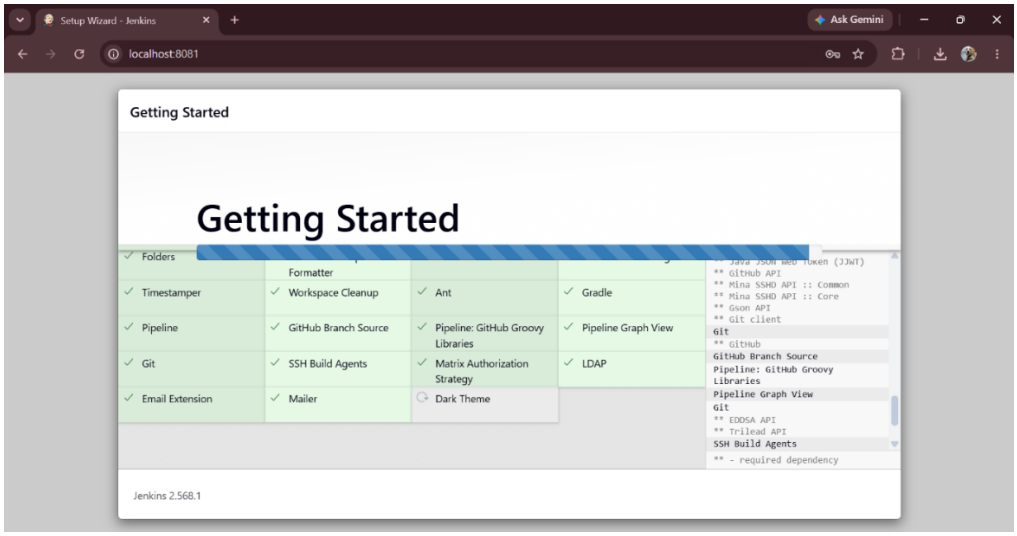
C:\Users\yogini bhatia>sc query jenkins

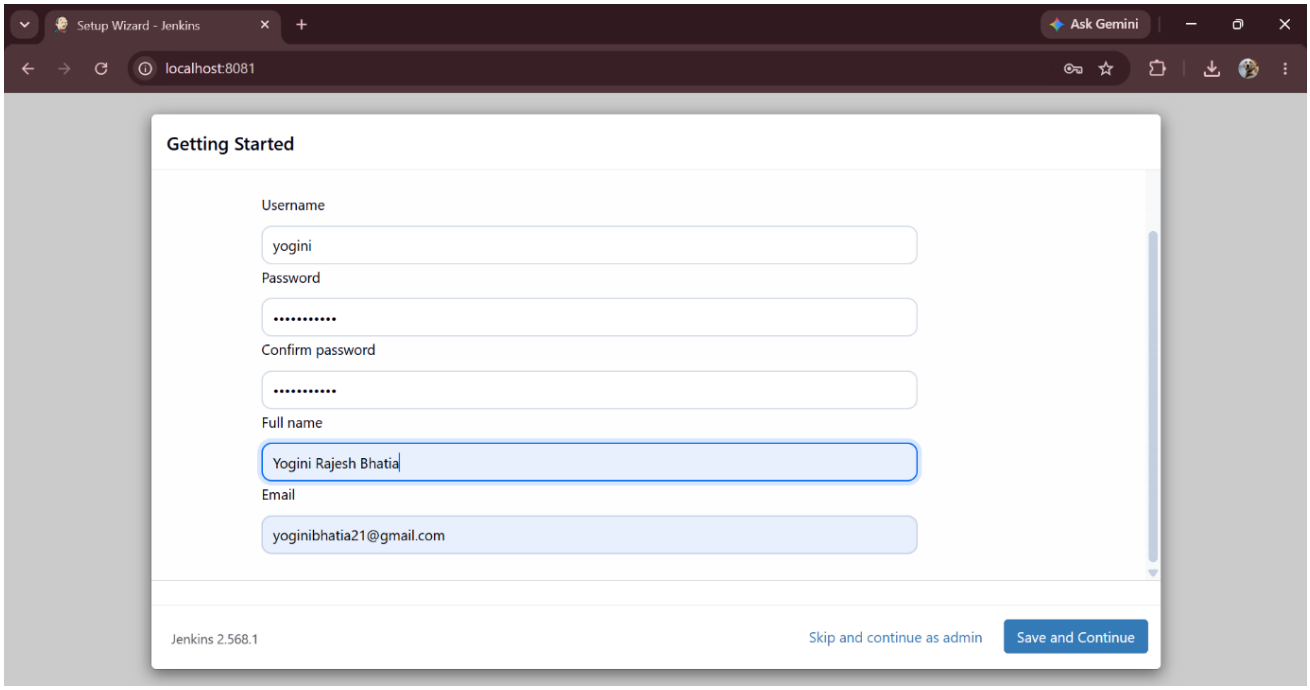
SERVICE_NAME: jenkins
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```




localhost:8081/login?from=%2F
Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password
Continue


Setup Wizard - Jenkins
Ask Gemini
localhost:8081
Getting Started
Customize Jenkins
Plugins extend Jenkins with additional features to support many different needs.
Install suggested plugins
Install plugins the Jenkins community finds most useful.
Select plugins to install
Select and install plugins most suitable for your needs.
Jenkins 2.568.1
ENG IN
18:29
03-08-2026


Setup Wizard - Jenkins
Ask Gemini
localhost:8081
Getting Started
Getting Started
Folders
Formatter
Timestamper
Workspace Cleanup
Ant
Gradle
Pipeline
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
Pipeline Graph View
Git
SSH Build Agents
Matrix Authorization Strategy
LDAP
Email Extension
Mailer
Dark Theme
Jenkins 2.568.1




```
C:\Users\yogini bhatia>cd Desktop

C:\Users\yogini bhatia\Desktop>mkdir JenkinsDemo

C:\Users\yogini bhatia\Desktop>cd JenkinsDemo

C:\Users\yogini bhatia\Desktop\JenkinsDemo>echo Hello Jenkins > hello.txt

C:\Users\yogini bhatia\Desktop\JenkinsDemo>type hello.txt
Hello Jenkins

C:\Users\yogini bhatia\Desktop\JenkinsDemo>notepad Jenkinsfile

C:\Users\yogini bhatia\Desktop\JenkinsDemo>dir
 Volume in drive C is Windows
 Volume Serial Number is 1C13-9CB9

 Directory of C:\Users\yogini bhatia\Desktop\JenkinsDemo
```

```
C:\Users\yogini bhatia\Desktop\JenkinsDemo>dir
 Volume in drive C is Windows
 Volume Serial Number is 1C13-9CB9

 Directory of C:\Users\yogini bhatia\Desktop\JenkinsDemo

03-08-2026  18:40    <DIR>          .
03-08-2026  18:37    <DIR>          ..
03-08-2026  18:37                16 hello.txt
03-08-2026  18:39               697 Jenkinsfile
               2 File(s)            713 bytes
               2 Dir(s)  272,609,611,776 bytes free
```

```
C:\Users\yogini bhatia\Desktop\JenkinsDemo>git init
Initialized empty Git repository in C:/Users/yogini bhatia/Desktop/JenkinsDemo/.git/

C:\Users\yogini bhatia\Desktop\JenkinsDemo>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Jenkinsfile
        hello.txt

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\yogini bhatia\Desktop\JenkinsDemo>git add .

C:\Users\yogini bhatia\Desktop\JenkinsDemo>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Jenkinsfile
        new file:   hello.txt


C:\Users\yogini bhatia\Desktop\JenkinsDemo>git commit -m "Initial Jenkins pipeline"
[master (root-commit) 5a0a81e] Initial Jenkins pipeline
 2 files changed, 38 insertions(+)
 create mode 100644 Jenkinsfile
 create mode 100644 hello.txt
```

```
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Jenkinsfile
        new file:   hello.txt


C:\Users\yogini bhatia\Desktop\JenkinsDemo>git commit -m "Initial Jenkins pipeline"
[master (root-commit) 5a0a81e] Initial Jenkins pipeline
 2 files changed, 38 insertions(+)
 create mode 100644 Jenkinsfile
 create mode 100644 hello.txt

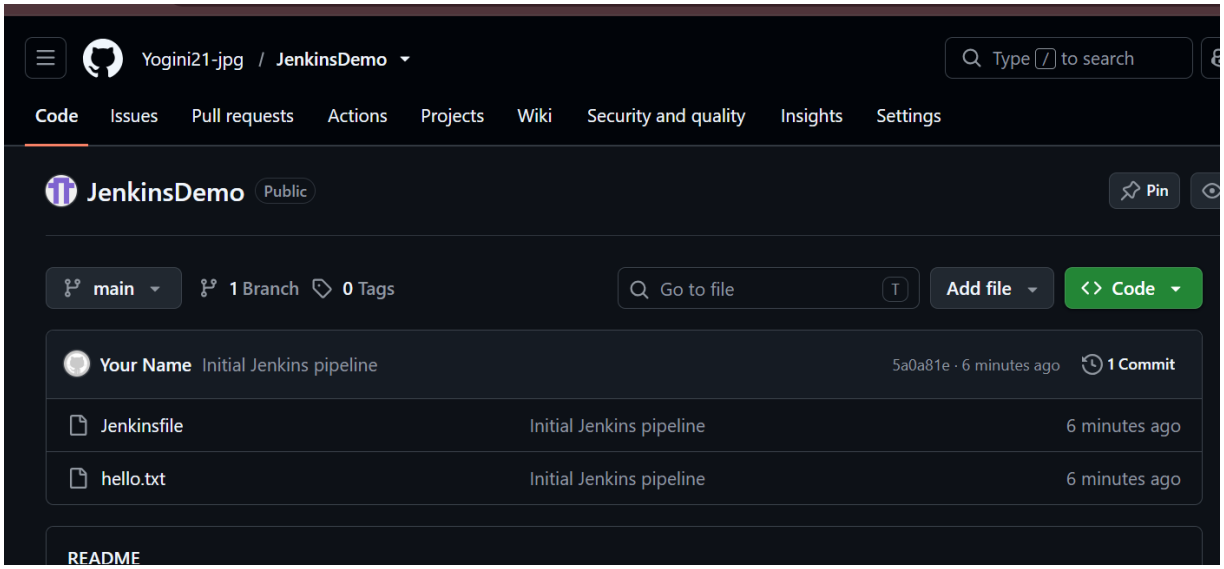
C:\Users\yogini bhatia\Desktop\JenkinsDemo>git remote add origin https://github.com/Yogini21-jpg/JenkinsDemo.git

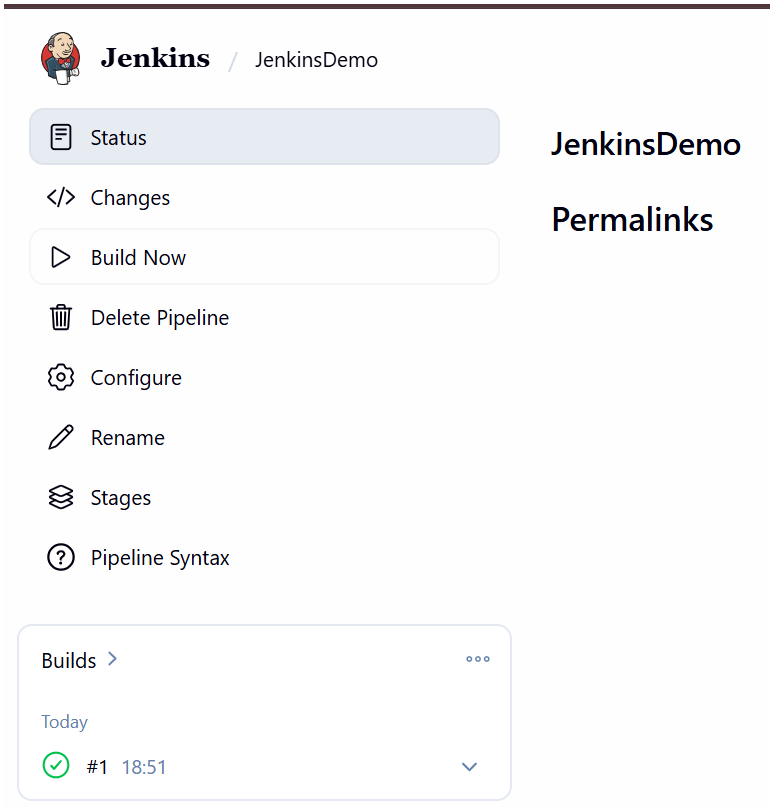
C:\Users\yogini bhatia\Desktop\JenkinsDemo>git remote -v
origin  https://github.com/Yogini21-jpg/JenkinsDemo.git (fetch)
origin  https://github.com/Yogini21-jpg/JenkinsDemo.git (push)

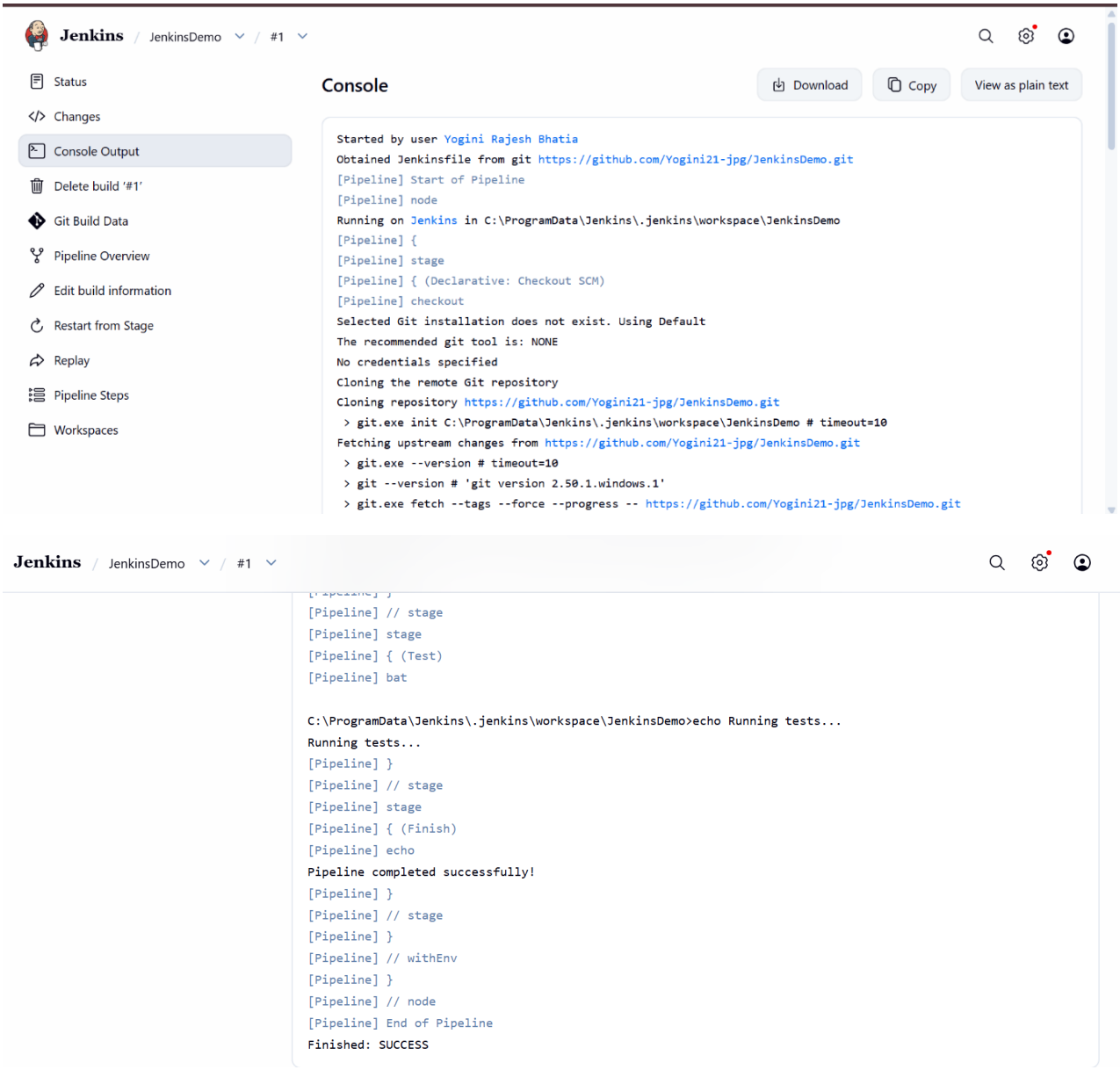
C:\Users\yogini bhatia\Desktop\JenkinsDemo>git branch -M main

C:\Users\yogini bhatia\Desktop\JenkinsDemo>git branch
* main

C:\Users\yogini bhatia\Desktop\JenkinsDemo>git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 482 bytes | 482.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Yogini21-jpg/JenkinsDemo.git
 * [new branch]      main -> main
```


Yogini21-jpg / JenkinsDemo
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
JenkinsDemo
Public
Pin
main
1 Branch
0 Tags
Go to file
T
Add file
Code
Your Name Initial Jenkins pipeline
5a0a81e · 6 minutes ago
1 Commit
Jenkinsfile
Initial Jenkins pipeline
6 minutes ago
hello.txt
Initial Jenkins pipeline
6 minutes ago
README


Jenkins / JenkinsDemo
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
Builds
Today
#1 18:51
JenkinsDemo
Permalinks







## CONCLUSION:

Thus, the Jenkins Pipeline was successfully created and executed. The experiment demonstrated the concept of Continuous Integration, Jenkins installation and setup, and the creation of a basic Declarative Pipeline.

The pipeline was structured using pipeline, agent, stages, stage, and steps blocks, and successfully executed the echo command to display “Hello, Jenkins!”. The experiment also provided an understanding of how Jenkins can automate software development processes and support CI/CD workflows.